2520090060
P.Vedadri Naidu

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
vedadripinninti@vedadris-MacBook-Air ~ %
```

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_TOKENS 100
int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int token_count;
    printf("myShell> ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';
    if (strlen(input) == 0) {
        printf("Empty command.\n");
        return 0;
    }
    token_count = 0;
```

```
    char *token = strtok(input, " \t");
    while (token != NULL && token_count < MAX_TOKENS - 1) {
        tokens[token_count++] = token;
        token = strtok(NULL, " \t");
    }
    tokens[token_count] = NULL;
    printf("\n--- Parsed Arguments ---\n");
    for (int i = 0; i < token_count; i++) {
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    }
    printf("\nToken count: %d\n", token_count);
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        execvp(tokens[0], tokens);
        perror("Command execution failed");
        exit(1);
    }
    else {
        printf("Parent PID: %d\n", getpid());
        wait(NULL);
        printf("Command execution completed.\n");
    }
    return 0;
}
```

OUTPUT

```
vedadripinninti@vedadris-MacBook-Air ~ % nano s4task1.c
vedadripinninti@vedadris-MacBook-Air ~ % gcc s4task1.c -o s4task
vedadripinninti@vedadris-MacBook-Air ~%./s4task1
```

```
myShell> ls
--- Parsed Arguments ---
Argument 1: ls
Token count: 1
Parent PID: 2118
Child PID: 12848
File2.txt command_executor.c cpro file.c filename process.c s4task1.c task3 task4.c week3
OSSP copyfile cpro.c file.c.save filename.c s4task task task3.c week1 week3.c
command_executor copyfile.c file file.txt process s4task1 task1.c task4 week1.c
Command execution completed.
```

Task-2

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
vedadripinninti@vedadris-MacBook-Air ~ %nano s4task2.c
```

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
```

```
#define MAX_TOKENS 100
int parseInput(char *input, char *tokens[]) {
    int count = 0;
    char *p = input;
    while (*p) {
        while (*p == ' ' || *p == '\t')
            p++;
        if (*p == '\0')
            break;
        char *token = malloc(MAX_INPUT);
        int i = 0;
        if (*p == '\'') {
            p++;
            while (*p != '\'' && *p != '\0') {
                token[i++] = *p++;
            }
            if (*p == '\0') {
                free(token);
                printf("Syntax Error: Unmatched single quote.\n");
                return -1;
            }
            p++;
        } else {
            while (*p != ' ' && *p != '\t' && *p != '\0') {
                token[i++] = *p++;
            }
        }
        token[i] = '\0';
        tokens[count++] = token;
```

```
    }
    tokens[count] = NULL;
    return count;
}
int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    printf("myShell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 0;
    input[strcspn(input, "\n")] = '\0';
    int token_count = parseInput(input, tokens);
    if (token_count == -1)
       return 1;
    if (token_count == 0) {
        printf("Empty command.\n");
        return 0;
    }
    printf("\nParsing Result:\n");
    for (int i = 0; i < token_count; i++)
        printf("Argument %d: %s\n", i + 1, tokens[i]);
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        printf("\nChild PID: %d\n", getpid());
        execvp(tokens[0], tokens);
```

```
        perror("Command execution failed");
        exit(1);
    } else {
        waitpid(pid, NULL, 0);
        printf("\nCommand execution completed.\n");
    }
    for (int i = 0; i < token_count; i++)
        free(tokens[i]);
    return 0;
}
```

```
vedadripinninti@vedadris-MacBook-Air ~  % nano s4task2.c
vedadripinninti@vedadris-MacBook-Air ~ % gcc s4task2.c -o s4task2
vedadripinninti@vedadris-MacBook-Air ~ ./s4task2
```

Output

```
vedadripinninti@vedadris-MacBook-Air ~ %
myShell> echo 'Hello World'
Parsing Result:
Argument 1: echo
Argument 2: Hello World
Child PID: 16348
Hello World
Command execution completed.
```

Task-3

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
vedadripinninti@vedadris-MacBook-Air ~ %nano skill4task3.c
vedadripinninti@vedadris-MacBook-Air ~ %gcc skill4task3.c -o skill4task3
```

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_TOKENS 100
int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;
    printf("myShell> ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';
    char *p = input;
    while (*p) {
        while (*p == ' ' || *p == '\t')
            p++;
        if (*p == '\0')
            break;
        char *token = malloc(MAX_INPUT);
        int i = 0;

        if (*p == '"') {
            p++;
            while (*p != '"' && *p != '\0') {
                token[i++] = *p++;
            }
            if (*p == '\0') {
```

```
            printf("Syntax Error: Unmatched double quote.\n");
            free(token);
            return 1;
        }
        p++;
    } else {
        while (*p != ' ' && *p != '\t' && *p != '\0')
            token[i++] = *p++;
    }
    token[i] = '\0';
    /* Variable expansion */
    char expanded[MAX_INPUT] = "";
    char *q = token;
    while (*q) {
        if (*q == '$') {
            q++;
            char var[100];
            int j = 0;
            while (*q && (isalnum(*q) || *q == '_'))
                var[j++] = *q++;
            var[j] = '\0';
            char *value = getenv(var);
            if (value)
                strcat(expanded, value);
        } else {
            int len = strlen(expanded);
            expanded[len] = *q++;
            expanded[len + 1] = '\0';
        }
```

```
    }
    strcpy(token, expanded);
    tokens[count++] = token;
}
tokens[count] = NULL;
if (count == 0) {
    printf("Empty command.\n");
    return 0;
}
printf("\n--- Parsed Arguments ---\n");
for (int i = 0; i < count; i++)
    printf("Argument %d: %s\n", i + 1, tokens[i]);
pid_t pid = fork();
if (pid == 0) {
    printf("\nChild Process\n");
    printf("PID = %d\n\n", getpid());
    execvp(tokens[0], tokens);
    perror("Execution failed");
    exit(1);
}
else if (pid > 0) {
    waitpid(pid, NULL, 0);
    printf("\nChild process completed.\n");
}
else {
    perror("fork");
}
for (int i = 0; i < count; i++)
    free(tokens[i]);
```

```
    return 0;
}
```

Output

```
vedadripinninti@vedadris-MacBook-Air ~ %
/skill4task3 myShell> echo "Hello $USER"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello root
Child Process
PID = 16478
Hello root
Child process completed.
```

Task-4

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
vedadripinninti@vedadris-MacBook-Air ~ % nano s4task4.c
```

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_INPUT 1024
#define MAX_TOKENS 100
int main() {
```

```
char input[MAX_INPUT];
char *tokens[MAX_TOKENS];
int count = 0;
printf("myShell> ");
fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';
char *p = input;
char token[MAX_INPUT];
int i = 0;
while (*p) {
    if (*p == '\\') {
        p++;
        if (*p == ' ' || *p == '\\' || *p == '"' ||
            *p == '\'' || *p == '$' || *p == '&' ||
            *p == '|' || *p == ';') {
            token[i++] = *p;
            p++;
        } else if (*p) {
            token[i++] = *p++;
        }
    }
    else if (*p == ' ') {
        if (i > 0) {
            token[i] = '\0';
            tokens[count++] = strdup(token);
            i = 0;
        }
        p++;
    }
```

```
        else {
            token[i++] = *p++;
        }
    }
    if (i > 0) {
        token[i] = '\0';
        tokens[count++] = strdup(token);
    }
    printf("\n--- Parsed Arguments ---\n");
    for (int j = 0; j < count; j++) {
        printf("Argument %d: %s\n", j + 1, tokens[j]);
        free(tokens[j]);
    }
    return 0;
}
```

Output

```
vedadripinninti@vedadris-MacBook-Air ~ % nano s4task4.c
vedadripinninti@vedadris-MacBook-Air ~ % gcc s4task4.c -o
s4task4 vedadripinninti@vedadris-MacBook-Air ~ %./s4task4
myShell> echo Hello\ World
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
vedadripinninti@vedadris-MacBook-Air ~ %nano s4task5.c
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100
int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    int count;
    while (1) {
        printf("myshell$ ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
            continue;
        if (strcmp(input, "exit") == 0)
            break;
        count = 0;
        char *token = strtok(input, " ");
        while (token != NULL && count < MAX_ARGS - 1) {
            args[count++] = token;
            token = strtok(NULL, " ");
        }
        args[count] = NULL;
        printf("Parent Process PID: %d\n", getpid());
        pid_t pid = fork();
        if (pid < 0) {
```

```
            perror("fork failed");
            continue;
        }
        if (pid == 0) {
            printf("Child Process PID: %d\n", getpid());
            execvp(args[0], args);
            perror(args[0]);
            exit(127);
        }
        else {
            printf("Created Child PID: %d\n", pid);

            int status;
            waitpid(pid, &status, 0);
            if (WIFEXITED(status)) {
                printf("Child exited with status: %d\n",
                        WEXITSTATUS(status));
            }
        }
    }
    return 0;
}
```

Output

```
vedadripinninti@vedadris-MacBook-Air ~ %nano s4task5.c
vedadripinninti@vedadris-MacBook-Air ~ %gcc s4task5.c -o s4task5
vedadripinninti@vedadris-MacBook-Air ~ %./s4task5
myshell$ ls
Parent Process PID: 641
```

```
Created Child PID: 642

Child Process PID: 642

File2.txt          copyfile  file        filename   s4task    s4task2.c  s4task4
skill4task3   task3   week1

OSSP               copyfile.c  file.c      filename.c  s4task1   s4task3   s4task4.c
skill4task3.c  task3.c  week1.c

command_executor   cpro       file.c.save  process    s4task1.c  s4task3.  s4task5
task          task4   week3

command_executor.c  cpro.c     file.txt     process.c  s4task2   s4task3.c  s4task5.c
task1.c       task4.c  week3.c

Child exited with status: 0

myshell$ pwd

Parent Process PID: 641

Created Child PID: 645

Child Process PID: 645

/root

Child exited with status: 0

myshell$ exit
```